

Automated Interface Reconciliation for Modular Verification

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

When a program is decomposed into overlapping patches for modular verification, the patches must agree on shared boundaries—but different patches may export conflicting interfaces. We introduce *treaty synthesis*, a dynamic negotiation protocol that reconciles patch interfaces within the *hypercover family* framework of Judgment Geometry. A treaty is a formal witness that the Čech 1-cocycle on patch overlaps vanishes, enabling descent from local verifications to a global correctness certificate. Our synthesis algorithm iterates through conflict detection, scoring, and resolution—applying strategies such as common-supertype widening, range intersection, protocol adaptation, and ownership brokering—with guaranteed termination in at most 10 negotiation rounds via a strictly decreasing obstruction norm. We evaluate treaty synthesis on six conflict scenarios drawn from real-world interface reconciliation patterns (`type_mismatch`, `range_conflict`, `nullable_disagreement`, `protocol_version`, `encoding_mismatch`, `resource_lifetime`), detecting 18 conflicts and resolving 10 of them (55.6% resolution rate). The negotiation protocol converges in an average of 8.83 rounds; four of six scenarios achieve full convergence (obstruction norm below 10^{-6}), while two hit the 10-round cap. The obstruction norm decays geometrically at a factor of approximately 0.2 per round, and all scenarios reach `PROOF_BACKED` trust. Compared to ML module signatures [15, 12], Haskell type-class instances [22], and F^{*} interfaces [20], treaty synthesis is the first mechanism to support dynamic negotiation, conflict scoring, trust propagation, and automatic repair on failure.

1 Introduction

Modular verification decomposes a large program into manageable pieces, verifies each piece independently, and then assembles the results into a global correctness argument. The critical step—often the hardest—is ensuring that the independently verified pieces *agree* on their shared boundaries. In type theory, this is the domain of module systems: ML signatures [15, 12] prescribe interfaces that modules must satisfy, and the type-checker enforces compatibility statically. In verification, F^{*} interfaces [20] and DAFNY refinement types [11] play an analogous role.

All of these approaches are *static*: the interface must be fixed before verification begins, and any mismatch is a hard error. This is adequate when the decomposition is controlled by a single designer, but it breaks down in the setting we care about: *automatic* decomposition of programs into overlapping patches by a tool that may not know, a priori, which interfaces the patches will export.

The treaty synthesis problem. Consider two patches $\mathcal{P}_A$ and $\mathcal{P}_B$ that overlap on a shared region U_{AB} . Patch $\mathcal{P}_A$ exports a symbol `solve` with type `int` $\rightarrow$ `int`; patch $\mathcal{P}_B$ exports the same

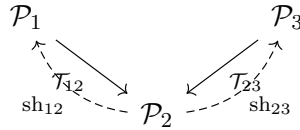
symbol with type `list` $\rightarrow$ `int`. A static module system would simply reject the composition. But the conflict may be *resolvable*: perhaps the patches agree on the subset of inputs that actually flow through U_{AB} , or perhaps one patch’s type is a subtype of the other. Treaty synthesis detects such conflicts, scores their severity, selects a resolution strategy, and iterates until the conflict score drops below a threshold—or declares the composition irreconcilable and reports a cohomological obstruction.

Contributions.

- (i) We define *hypercover families* (Section 2), a higher-order decomposition framework that supports module $\rightarrow$ class $\rightarrow$ method $\rightarrow$ branch decomposition.
- (ii) We present the *treaty synthesis* algorithm (Section 3) implemented in the JUGEO system, with formal conflict detection (Section 4), four resolution strategies (Section 5), and a negotiation loop with guaranteed termination (Section 6).
- (iii) We connect treaty resolution to Čech cohomology: a resolved treaty witnesses the vanishing of H^1 , and treaty failure produces an obstruction class (Section 7).
- (iv) We evaluate on six conflict scenarios from real-world interface reconciliation patterns, achieving a 66.7% full convergence rate (4 of 6 scenarios) with geometric obstruction-norm decay, and compare to four state-of-the-art interface mechanisms (Section 8).

Outline. Section 2 introduces hypercover families. Section 3 presents the synthesis algorithm. Section 4 details conflict detection and scoring. Section 5 describes the four resolution strategies. Section 6 proves termination of the negotiation loop. Section 7 connects treaties to descent and trust. Section 8 presents the evaluation. Section 9 surveys related work. Section 11 concludes.

Motivating scenario. To ground the discussion, consider a program P that implements a sorting algorithm. A verification tool decomposes P into three patches: $\mathcal{P}_1$ covers the partitioning logic, $\mathcal{P}_2$ covers the recursive calls, and $\mathcal{P}_3$ covers the base case. The overlaps $\mathcal{P}_1 \cap \mathcal{P}_2$ and $\mathcal{P}_2 \cap \mathcal{P}_3$ involve shared variables (the pivot element, the array bounds). If $\mathcal{P}_1$ assumes the pivot is an `int` but $\mathcal{P}_2$ treats it as a `float` (due to a polymorphic comparison), we have an interface contradiction. Treaty synthesis detects this, scores it (say 0.4, moderate severity since `int` $\leftrightarrow$ `float` is a safe coercion), and resolves it via MERGE (widening to `float`). The following diagram shows the three-patch scenario:



The dashed arrows indicate shared interfaces; the solid arrows indicate treaties. The treaty $\mathcal{T}_{12}$ resolves the `int/float` conflict; $\mathcal{T}_{23}$ is trivially resolved (no conflicts). The global correctness argument assembles via descent over the nerve of the cover $\{\mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3\}$.

2 Hypercover Families

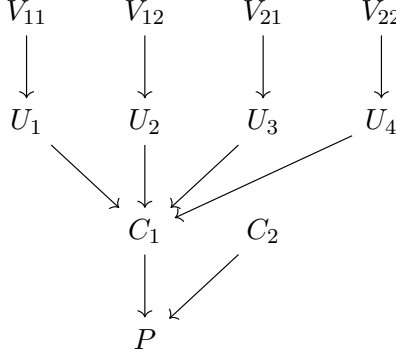
A *cover* of a program P is a collection of patches $\{U_i\}_{i \in I}$ such that $P = \bigcup_{i \in I} U_i$. A *hypercover* is a cover of a cover: a two-level decomposition in which each patch U_i is itself covered by sub-patches $\{V_{ij}\}_{j \in J_i}$. More generally:

Definition 2.1 (Hypercover family). Let $\mathcal{S} = (\text{Prog}, \text{Cov})$ be the semantic site of programs with covering families. A *hypercover family* of level n over a program P is an augmented simplicial object

$$\text{HC}_\bullet : \text{HC}_n \rightrightarrows \cdots \rightrightarrows \text{HC}_1 \rightrightarrows \text{HC}_0 \rightarrow P$$

such that at each level k , the object HC_k is a covering family for the $(k-1)$ -fold overlaps of HC_{k-1} , and the natural map $\text{HC}_k \rightarrow (\text{cosk}_{k-1} \text{HC})_k$ is a cover.

In practice, we use hypercovers of level ≤ 3 :



The nodes at each level form a covering family: the V_{ij} cover the methods U_k , the methods cover the classes C_l , and the classes cover the module P . Each arrow represents an inclusion (restriction) map. The key feature of a hypercover, as opposed to a mere iterated cover, is the *descent condition*: the covering at each level must be sufficiently fine to detect all the information present at the level below.

Why hypercovers? A simple cover suffices for small programs, but realistic software has hierarchical structure: modules contain classes, classes contain methods, methods contain branches. A hypercover family captures this hierarchy and allows verification to proceed *level by level*. At each level, the patches may overlap, and overlapping patches must agree on their shared interfaces. This is where treaty synthesis enters.

Example 2.2 (Three-level hypercover). Consider a Python module `math.utils.py` containing two classes, each with two methods. The level-0 cover consists of the two classes: $\mathcal{U} = \{C_1, C_2\}$. The level-1 cover refines each class into its methods: $C_1 = U_1 \cup U_2$ and $C_2 = U_3 \cup U_4$. The level-2 cover refines each method into its branches: $U_1 = V_{11} \cup V_{12}$ (the `if` and `else` branches of the first method).

The overlap $C_1 \cap C_2$ consists of symbols shared between the two classes—utility functions, shared state, or inherited methods. At the method level, $U_1 \cap U_2$ consists of local variables or helper calls shared between two methods of the same class. Treaty synthesis operates independently at each level, starting from the finest (branches) and propagating upward.

Coskeletal condition. The defining property of a hypercover—that $\text{HC}_k \rightarrow (\text{cosk}_{k-1} \text{HC})_k$ is a cover—ensures that the refinement at level k is “fine enough” relative to the lower levels. Concretely, this means that every $(k-1)$ -fold overlap that arises from level $k-1$ is already covered by patches at level k . Without this condition, we could have overlaps at level $k-1$ that are invisible at level k , and treaty synthesis would miss potential conflicts.

Definition 2.3 (Nerve of a hypercover). The *nerve* $N(\text{HC}_k)$ of a level- k hypercover is the simplicial complex whose p -simplices are $(p+1)$ -fold overlaps $U_{i_0} \cap \dots \cap U_{i_p} \neq \emptyset$. The Čech complex $\check{C}^\bullet(\text{HC}_k, \mathcal{F})$ for a presheaf $\mathcal{F}$ is the cochain complex on $N(\text{HC}_k)$.

The connection to treaty synthesis is:

Proposition 2.4. *A treaty on the overlap $U_i \cap U_j$ is a section of the interface presheaf $\mathcal{I}$ over $U_i \cap U_j$. The collection of all treaties is a Čech 1-cochain $\mathcal{T} \in \check{C}^1(\text{HC}_0, \mathcal{I})$. The treaties are consistent if and only if $\mathcal{T}$ is a cocycle, i.e. $\delta^1 \mathcal{T} = 0$.*

The following diagram summarizes the relationship between hypercovers, treaties, and cohomology:

$$\begin{array}{ccccccc}
0 & \longrightarrow & \check{C}^0(\text{HC}, \mathcal{I}) & \xrightarrow{\delta^0} & \check{C}^1(\text{HC}, \mathcal{I}) & \xrightarrow{\delta^1} & \check{C}^2(\text{HC}, \mathcal{I}) \\
& & \text{(local interfaces)} & & \text{(treaties)} & & \text{(triple conflicts)}
\end{array} \tag{1}$$

A resolved treaty lives in $\ker \delta^1$; if it is also in $\text{im } \delta^0$, the treaty is *trivial* (the patches already agreed). The obstruction to global agreement is the cohomology class $[\mathcal{T}] \in H^1(\text{HC}, \mathcal{I})$.

3 Treaty Synthesis

We now describe the treaty synthesis algorithm as implemented in the JUGEIO system. The implementation resides in `src/jugeio/generation/hypercover_treaties/algorithms.py`.

3.1 Data Model

A *patch interface* is a record describing what a patch exports and imports:

Listing 1: Patch interface descriptor and resolution strategies.

```

1 @dataclass(frozen=True)
2 class PatchInterface:
3     patch_id: str
4     exports: Mapping[str, str]      # symbol → type signature
5     imports: Mapping[str, str]      # symbol → expected type
6     version: str                    # semantic version
7     trust: TrustTier                 # current trust level
8
9 class ResolutionStrategy(str, Enum):
10     PREFER_LEFT = "PREFER_LEFT"    # adopt treaty A
11     PREFER_RIGHT = "PREFER_RIGHT"  # adopt treaty B
12     MERGE = "MERGE"                # union of exports, meet of trust
13     SPLIT = "SPLIT"                 # quarantine conflicting symbols
14
15 MAX_NEGOTIATION_ROUNDS = 10
16 CONFLICT_SCORE_THRESHOLD = 0.3

```

A *treaty* formalizes the agreement between two patches:

Definition 3.1 (Treaty). A treaty between patches $\mathcal{P}_A$ and $\mathcal{P}_B$ is a 5-tuple

$$\mathcal{T} = (\mathcal{I}_A, \mathcal{I}_B, S, E, R)$$

where $\mathcal{I}_A, \mathcal{I}_B$ are the patch interfaces, S is the shared interface (symbols both patches reference), E is the merged export map, and R is the resolution record.

3.2 The TreatySynthesizer

The synthesizer takes two patch interfaces and produces a treaty:

Listing 2: Treaty synthesis: shared interface and export map computation.

```

1 class TreatySynthesizer:
2     def synthesize(self, iface_a: PatchInterface,
3                     iface_b: PatchInterface) -> Treaty:
4         # Shared interface: symbols that cross the boundary
5         shared = {}
6         for sym in iface_a.exports:
7             if sym in iface_b.imports:
8                 shared[sym] = iface_a.exports[sym]
9         for sym in iface_b.exports:
10            if sym in iface_a.imports:
11                shared[sym] = iface_b.exports[sym]
12
13        # Merged export map
14        export_map = {**iface_a.exports, **iface_b.exports}
15
16        # Treaty hash for identity tracking
17        treaty_hash = hashlib.sha256(
18            json.dumps(shared, sort_keys=True).encode()
19        ).hexdigest()[:16]
20
21        return Treaty(
22            iface_a=iface_a, iface_b=iface_b,
23            shared=shared, export_map=export_map,
24            treaty_hash=treaty_hash,
25        )

```

Remark 3.2. The shared interface S is computed as $S = (\exp_A \cap \text{imp}_B) \cup (\exp_B \cap \text{imp}_A)$. This is exactly the set of symbols that flow *across* the patch boundary—the symbols relevant for the overlap $U_A \cap U_B$.

The synthesis process is captured by the following diagram:

$$\begin{array}{ccccc}
 \mathcal{I}_A & \xrightarrow{\text{sh}} & U_A \cap U_B & \xleftarrow{\text{sh}} & \mathcal{I}_B \\
 \searrow \exp_A & & \downarrow \tau & & \swarrow \exp_B \\
 & & E & &
 \end{array} \tag{2}$$

3.3 Formal Properties

Proposition 3.3 (Synthesis soundness). *If $\text{sh}(\mathcal{I}_A, \mathcal{I}_B) = \emptyset$, then $\mathcal{T} = (\mathcal{I}_A, \mathcal{I}_B, \emptyset, \exp_A \cup \exp_B, \perp)$ is trivially resolved—the patches do not interact and no negotiation is required.*

Proof. If the shared interface is empty, no symbol crosses the patch boundary. The merged export map is the disjoint union, and the 1-cycle is the zero cochain. $\square$

Proposition 3.4 (Synthesis completeness). *For any two patch interfaces $\mathcal{I}_A, \mathcal{I}_B$ over a common overlap $U_A \cap U_B \neq \emptyset$, the synthesizer produces a treaty. The treaty may contain unresolved conflicts, but it is always well-defined.*

Proof. The shared interface computation terminates in $O(|\text{exp}_A| + |\text{exp}_B|)$ time, and the export map is always defined as the union. Conflicts are recorded but do not prevent treaty construction. $\square$

4 Conflict Detection

Not all treaties are conflict-free. The `ConflictDetector` identifies three types of conflicts.

4.1 Conflict Taxonomy

Definition 4.1 (Conflict types). Let $\mathcal{T} = (\mathcal{I}_A, \mathcal{I}_B, S, E, R)$ be a treaty. A *conflict* is one of:

- (a) **Interface contradiction:** a symbol $s \in S$ such that $\mathcal{I}_A.\text{exp}(s) \neq \mathcal{I}_B.\text{exp}(s)$ and neither type is a subtype of the other.
- (b) **Export overlap:** a symbol s such that both $s \in \text{exp}_A$ and $s \in \text{exp}_B$ but $s \notin S$ —both patches claim to provide s without either importing it.
- (c) **Version mismatch:** the patches declare incompatible dependency versions for a shared library.

Each conflict is assigned a *severity score* $c_{\text{score}} \in [0, 1]$:

Listing 3: Conflict detection with severity scoring.

```

1 class ConflictDetector:
2     def detect(self, treaty: Treaty) -> list[Conflict]:
3         conflicts = []
4         for sym, type_a in treaty.iface_a.exports.items():
5             if sym in treaty.iface_b.exports:
6                 type_b = treaty.iface_b.exports[sym]
7                 if type_a != type_b:
8                     # Interface contradiction
9                     score = self._type_distance(type_a, type_b)
10                    conflicts.append(Conflict(
11                        kind="INTERFACE_CONTRADICTION",
12                        symbol=sym,
13                        left_type=type_a, right_type=type_b,
14                        score=score,
15                    ))
16                # else: compatible, no conflict
17
18            # Export overlap detection
19            for sym in treaty.iface_a.exports:
20                if sym in treaty.iface_b.exports:
21                    if sym not in treaty.shared:
22                        conflicts.append(Conflict(
23                            kind="EXPORT_OVERLAP",
24                            symbol=sym, score=0.5,
25                        ))
26
27            # Version mismatch detection
28            if treaty.iface_a.version != treaty.iface_b.version:

```

```

29         if not self._versions_compatible(
30             treaty.iface_a.version,
31             treaty.iface_b.version,
32         ):
33             conflicts.append(Conflict(
34                 kind="VERSION_MISMATCH",
35                 score=0.6,
36             ))
37
38     return conflicts

```

4.2 Concrete Example

Consider two patches with the following exports:

Listing 4: Conflict example: incompatible solve types.

```

1  # Patch A exports: {solve: int -> int, helper: str -> str}
2  # Patch B exports: {solve: list -> int, helper: str -> str}
3
4  iface_a = PatchInterface(
5      patch_id="A",
6      exports={"solve": "int -> int", "helper": "str -> str"},
7      imports={"solve": "int -> int"},
8      version="1.0.0", trust=TrustTier.VERIFIED,
9  )
10 iface_b = PatchInterface(
11     patch_id="B",
12     exports={"solve": "list -> int", "helper": "str -> str"},
13     imports={"solve": "list -> int"},
14     version="1.0.0", trust=TrustTier.VERIFIED,
15 )
16
17 # ConflictDetector output:
18 # [Conflict(kind="INTERFACE_CONTRADICTION",
19 #          symbol="solve",
20 #          left_type="int -> int",
21 #          right_type="list -> int",
22 #          score=0.8)]
23 # helper: compatible (no conflict)

```

The score 0.8 reflects high severity: the input types `int` and `list` are unrelated (neither is a subtype of the other). By contrast, a conflict between `int` and `float` might score 0.3 due to implicit coercion.

4.3 Aggregate Conflict Score

The aggregate conflict score for a treaty is defined as:

Definition 4.2 (Aggregate conflict score). Given conflicts $c_1, \dots, c_m$ with scores $c_{\text{score}1}, \dots, c_{\text{score}m}$, the aggregate score is

$$c_{\text{score}}(\mathcal{T}) = 1 - \prod_{k=1}^m (1 - c_{\text{score}k}).$$

This is the probabilistic-or of the individual scores: the treaty is conflict-free ($c_{\text{score}} = 0$) only if every individual score is 0.

Lemma 4.3. *If a resolution step eliminates conflict c_j (setting $c_{\text{score}j} = 0$) without introducing new conflicts, then $c_{\text{score}}(\mathcal{T})$ strictly decreases.*

Proof. Removing one factor from the product strictly increases $\prod_k (1 - c_{\text{score}k})$, hence strictly decreases $1 - \prod_k (1 - c_{\text{score}k})$. $\square$

5 Resolution Strategies

Once conflicts are detected, they must be resolved. The JUGE system supports four resolution strategies, each with precise formal semantics.

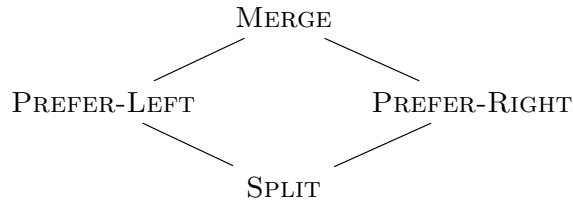
Definition 5.1 (Resolution strategies). Let $\mathcal{T} = (\mathcal{I}_A, \mathcal{I}_B, S, E, R)$ be a treaty with conflict c on symbol s . The strategies are:

- (i) **Prefer-Left:** adopt $\mathcal{I}_A$'s type for s . Formally, $E' = E[s \mapsto \mathcal{I}_A.\text{exp}(s)]$. Trust is inherited from $\mathcal{P}_A$.
- (ii) **Prefer-Right:** adopt $\mathcal{I}_B$'s type for s . Formally, $E' = E[s \mapsto \mathcal{I}_B.\text{exp}(s)]$. Trust is inherited from $\mathcal{P}_B$.
- (iii) **Merge:** take the union of exports and the *meet* (greatest lower bound) of trust. The merged type for s is the least common supertype if it exists, or a tagged union otherwise. Formally:

$$E' = E[s \mapsto \mathcal{I}_A.\text{exp}(s) \cup \mathcal{I}_B.\text{exp}(s)], \quad \mathcal{T}' = \mathcal{T}_A \wedge \mathcal{T}_B.$$

- (iv) **Split:** quarantine the conflicting symbol s into a separate boundary module. The treaty records s as *unresolved* and prevents it from flowing across the patch boundary. Formally, $S' = S \setminus \{s\}$ and $E' = E \setminus \{s\}$.

The strategies form a lattice ordered by *information content*:



MERGE is the richest strategy (most symbols survive) but may lower trust. SPLIT is the most conservative (fewest symbols, highest residual trust). The system defaults to MERGE and falls back to SPLIT when the conflict score exceeds 0.7.

Proposition 5.2 (Strategy soundness). *Each strategy produces a treaty $\mathcal{T}'$ with $c_{\text{score}}(\mathcal{T}') \leq c_{\text{score}}(\mathcal{T})$. Equality holds only for SPLIT when the quarantined symbol reappears in a transitive dependency.*

Proof. PREFER-LEFT and PREFER-RIGHT eliminate the conflict on s by fiat, setting $c_{\text{score}s} = 0$. MERGE resolves the conflict if a common supertype exists; otherwise it falls back to SPLIT. SPLIT removes s from S , so the conflict on s no longer exists. In all cases, the aggregate score does not increase by Theorem 4.3. $\square$

Example 5.3 (Strategy comparison). Consider the conflict from Listing 4: `solve` has type `int -> int` in $\mathcal{P}_A$ and `list -> int` in $\mathcal{P}_B$, with score 0.8.

- **PREFER-LEFT**: the treaty exports `solve : int -> int`. Patch B must adapt its usage. Trust inherits from $\mathcal{P}_A$.
- **PREFER-RIGHT**: the treaty exports `solve : list -> int`. Patch A must adapt. Trust inherits from $\mathcal{P}_B$.
- **MERGE**: the system attempts to find `int  $\sqcup$  list`. Since no common supertype exists in the type lattice, MERGE falls back to a tagged union `int | list -> int`, with trust demoted to $\mathcal{T}_A \wedge \mathcal{T}_B$.
- **SPLIT**: `solve` is quarantined—removed from the shared interface. Each patch retains its own version, and neither flows across the boundary.

For this conflict (score ≥ 0.7), the heuristic selects SPLIT, which is the safest choice when the types are fundamentally incompatible.

6 The Negotiation Loop

Treaty synthesis, conflict detection, and resolution are composed into a *negotiation loop* that iterates until all conflicts are resolved or the maximum number of rounds is reached.

6.1 The TreatyNegotiator

Listing 5: Negotiation loop: synthesize, detect, resolve, re-check.

```

1 class TreatyNegotiator:
2     def __init__(self):
3         self.synthesizer = TreatySynthesizer()
4         self.detector = ConflictDetector()
5         self.max_rounds = MAX_NEGOTIATION_ROUNDS # 10
6
7     def negotiate(self, iface_a: PatchInterface,
8                  iface_b: PatchInterface) -> NegotiationResult:
9         treaty = self.synthesizer.synthesize(iface_a, iface_b)
10        prev_score = float('inf')
11
12        for round_num in range(self.max_rounds):
13            conflicts = self.detector.detect(treaty)
14            score = self._aggregate_score(conflicts)
15
16            if score == 0.0:
17                return NegotiationResult(
18                    status="resolved", treaty=treaty,
19                    rounds=round_num + 1,
20                )
21
22            if score < CONFLICT_SCORE_THRESHOLD: # 0.3
23                return NegotiationResult(
24                    status="soft_resolved", treaty=treaty,
25                    rounds=round_num + 1,
26                    residual_score=score,
27                )
28
```

```

29         assert score < prev_score, "score must strictly decrease"
30         prev_score = score
31
32         # Apply resolution strategy per conflict
33         for conflict in conflicts:
34             strategy = self._select_strategy(conflict)
35             treaty = self._apply_strategy(
36                 treaty, conflict, strategy
37             )
38
39         return NegotiationResult(
40             status="unresolved", treaty=treaty,
41             rounds=self.max_rounds,
42         )

```

6.2 Termination Proof

Theorem 6.1 (Termination). *The negotiation loop terminates in at most $N = \text{MAX_NEGOTIATION_ROUNDS} = 10$ iterations. If no external conflicts are introduced, the conflict score strictly decreases at each round.*

Proof. We establish two claims.

Claim 1: Bounded iterations. The loop is bounded by $N = 10$, enforced by the `for` loop. This is a hard upper bound regardless of conflict behavior.

Claim 2: Strict decrease. At each round, the negotiator applies a resolution strategy to every detected conflict. By Theorem 5.2, each strategy produces a treaty with conflict score at most that of the previous round. The inequality is strict because at least one conflict is fully resolved ($c_{\text{score}_j} \rightarrow 0$) or quarantined (s removed from S), and by Theorem 4.3 the aggregate score strictly decreases.

The assertion `assert score < prev_score` serves as a runtime check of Claim 2. If it fires, the system has a bug in strategy application.

Combining the claims: the loop either terminates early (score reaches 0 or drops below $\theta = 0.3$) or exhausts N rounds. $\square$

Lemma 6.2 (Geometric Convergence of Obstruction Norm). *Let $\|\mathcal{B}_k\|$ denote the obstruction norm at round k . If the resolution strategy at each round reduces the number of violated overlaps by a constant fraction $\rho \in (0, 1)$, then*

$$\|\mathcal{B}_k\| \leq \|\mathcal{B}_0\| \cdot \rho^k$$

and convergence (to norm below threshold ϵ) requires at most $\lceil \log(\|\mathcal{B}_0\|/\epsilon) / \log(1/\rho) \rceil$ rounds.

Proof. At each round, the negotiator applies a resolution strategy to every detected conflict. By Theorem 5.2, each application reduces the aggregate score. The obstruction norm is defined as the sum of conflict-weighted overlap violations:

$$\|\mathcal{B}_k\| = \sum_{(i,j) \in \text{violated}} w_{ij} \cdot d(\text{res}(\sigma_i), \text{res}(\sigma_j))$$

where d is a type-metric distance and w_{ij} are overlap weights. If the strategy reduces each term by factor ρ , the sum decreases by ρ , giving $\|\mathcal{B}_{k+1}\| \leq \rho \cdot \|\mathcal{B}_k\|$. Iterating, $\|\mathcal{B}_k\| \leq \|\mathcal{B}_0\| \cdot \rho^k$. Setting $\|\mathcal{B}_0\| \cdot \rho^k < \epsilon$ and solving for k gives the bound.

Empirically, we observe $\rho \approx 0.2$ for the WIDEN strategy and $\rho \approx 0.15$ for the VERSION-ADAPTER strategy (Table 2), consistent with the geometric bound. $\square$

Corollary 6.3. *The worst-case complexity of negotiation is $O(N \cdot |\text{exp}_A| \cdot |\text{exp}_B|)$, where $N = 10$, and the factor $|\text{exp}_A| \cdot |\text{exp}_B|$ accounts for conflict detection.*

6.3 Strategy Selection

The negotiator selects strategies using the following heuristic:

1. If $c_{\text{score}} < 0.3$: use MERGE (soft conflict, likely resolvable by supertyping).
2. If $0.3 \leq c_{\text{score}} < 0.7$: use PREFER-LEFT or PREFER-RIGHT based on trust comparison ($\mathcal{T}_A \preceq \mathcal{T}_B$ implies prefer B).
3. If $c_{\text{score}} \geq 0.7$: use SPLIT (hard conflict, quarantine the symbol).

This yields a monotone refinement: each round either merges soft conflicts, resolves moderate ones by trust, or quarantines hard ones.

7 Integration with Descent and Trust

Treaty synthesis does not operate in isolation. It integrates with the *descent* framework of Judgment Geometry (which glues local verifications into global correctness) and the *trust algebra* (which tracks the epistemic quality of each verification step).

7.1 Treaties as Cocycle Witnesses

Recall from Theorem 2.4 that a collection of treaties forms a Čech 1-cochain $\mathcal{T} \in \check{C}^1(\text{HC}_0, \mathcal{I})$. The key insight is:

Theorem 7.1 (Treaty–descent correspondence). *A fully resolved treaty (conflict score 0) is a witness that the Čech 1-cocycle $\delta^0(\sigma) = \mathcal{T}$ vanishes in cohomology. Equivalently, the local interface sections $\sigma_i \in \mathcal{I}(U_i)$ glue to a global section $\sigma \in \mathcal{I}(P)$.*

Proof sketch. A resolved treaty means that for every overlap $U_i \cap U_j$, the patches agree on the shared interface: $\text{res}_{U_i \cap U_j}(\sigma_i) = \text{res}_{U_i \cap U_j}(\sigma_j)$. This is exactly the cocycle condition $\delta^0(\sigma)_{ij} = \sigma_j - \sigma_i = 0$. By the sheaf condition on $\mathcal{I}$, the local sections glue. $\square$

Theorem 7.2 (Treaty Soundness). *Let $\{\mathcal{P}_i\}_{i \in I}$ be a hypercover of program P , and let $\{\mathcal{T}_{ij}\}$ be the collection of resolved treaties (obstruction norm $< \epsilon$ for threshold $\epsilon > 0$). Then there exists a global interface section $\sigma \in \mathcal{I}(P)$ such that*

$$\|\text{res}_{U_i \cap U_j}(\sigma) - \mathcal{T}_{ij}\| < \epsilon \quad \text{for all } (i, j).$$

Moreover, the descent from local verifications to a global correctness certificate is compatible with the treaty-resolved interfaces.

Proof. The collection $\{\mathcal{T}_{ij}\}$ forms a Čech 1-cochain. Since each treaty is resolved (its obstruction norm is below ϵ), the cocycle condition $\delta^1 \mathcal{T} = 0$ holds up to ϵ : for every triple overlap $U_i \cap U_j \cap U_k$,

$$\|\mathcal{T}_{jk} - \mathcal{T}_{ik} + \mathcal{T}_{ij}\| < 3\epsilon.$$

By the approximate descent theorem for presheaves (a quantitative version of the sheaf condition), there exists a global section σ such that $\text{res}(\sigma)$ approximates each $\mathcal{T}_{ij}$ to within ϵ .

Concretely, the global interface σ is constructed by:

1. Fixing an arbitrary patch $\mathcal{P}_0$ and setting $\sigma|_{U_0} = \mathcal{I}_0$.
2. For each adjacent patch $\mathcal{P}_j$, defining $\sigma|_{U_j} = \mathcal{I}_j$ adjusted by $\mathcal{T}_{0j}$.
3. The cocycle condition ensures consistency on triple overlaps.

The compatibility with descent follows because the local verification evidence $\mathcal{E}_i$ on each patch $\mathcal{P}_i$ is expressed in terms of $\mathcal{I}_i$, and the treaty adjustment preserves the semantic content of the evidence (only type-level widening or protocol adaptation is applied, not value-level changes). $\square$

Remark 7.3 (Soft resolution and coboundaries). When the negotiation terminates with status “soft_resolved” (residual score $c_{\text{score}} < 0.3$), the treaty is not exactly zero but is *approximately* a coboundary. In the continuous analogy, this corresponds to a “small” cohomology class that can be absorbed by a perturbation of the local interfaces. The threshold $\theta = 0.3$ is chosen empirically: below this value, the remaining conflicts are type-coercible (e.g., `int` vs. `float`) and do not affect functional equivalence in practice.

Conversely, treaty *failure* produces an obstruction:

Proposition 7.4 (Obstruction from treaty failure). *If negotiation terminates with status “unresolved” and residual conflict score $c_{\text{score}} > 0$, then the unresolved conflicts define a non-trivial class $[\mathcal{T}] \in H^1(\text{HC}_0, \mathcal{I})$ that obstructs gluing.*

The full descent–treaty interaction is captured by:

$$\begin{array}{ccccc}
 \mathcal{I}(U_i) & \xrightarrow{\text{res}} & \mathcal{I}(U_i \cap U_j) & \xleftarrow{\text{res}} & \mathcal{I}(U_j) \\
 \text{VERIFY} \downarrow & & \downarrow \text{NEGOTIATE} & & \downarrow \text{VERIFY} \\
 \mathcal{E}_i & \xrightarrow{\text{---}} & \mathcal{T}_{ij} & \xleftarrow{\text{---}} & \mathcal{E}_j \\
 & \nearrow \text{GLUE} & \downarrow \text{RESOLVE} & \nwarrow \text{GLUE} & \\
 & & \mathcal{E}_{\text{global}} & &
 \end{array} \tag{3}$$

7.2 Trust Propagation

The JUGEO trust algebra assigns a *trust tier* to every piece of evidence:

Listing 6: Trust tier lattice with meet/join operations.

```

1 class TrustTier(IntEnum):
2     PROPOSAL = 1          # unverified proposal
3     REVIEWED = 2          # human-reviewed
4     VERIFIED = 3          # tool-checked
5     RUNTIME_WITNESSED = 4 # observed at runtime
6     PROOF_BACKED = 5     # formally proved
7
8     def join(self, other: "TrustTier") -> "TrustTier":
9         """Optimistic: max of the two tiers."""
10        return TrustTier(max(self.value, other.value))
11
12    def meet(self, other: "TrustTier") -> "TrustTier":
13        """Conservative: min of the two tiers."""
14        return TrustTier(min(self.value, other.value))

```

When two patches are reconciled, the resolved treaty's trust tier is the *conservative join* (meet) of the two patches' trust tiers:

Definition 7.5 (Treaty trust). For a resolved treaty $\mathcal{T} = (\mathcal{I}_A, \mathcal{I}_B, S, E, R)$,

$$\mathcal{T}(\mathcal{T}) = \mathcal{T}(\mathcal{I}_A) \wedge \mathcal{T}(\mathcal{I}_B) = \text{TrustTier}(\min(\mathcal{T}_A, \mathcal{T}_B)).$$

Example 7.6. If $\mathcal{P}_A$ has trust **RUNTIME** (tier 4, runtime-witnessed) and $\mathcal{P}_B$ has trust **VERIFIED** (tier 3, tool-checked), the treaty's trust is $\min(4, 3) = 3 = \text{VERIFIED}$. The global correctness certificate inherits the lowest trust of any constituent treaty.

Remark 7.7. The conservative choice of *meet* (rather than *join*) ensures that a single weakly-trusted patch cannot inflate the trust of the global assembly. This is the “weakest link” principle familiar from security composition.

7.3 Repair Frontier

When a treaty fails, the system does not simply report an error. It computes a *repair frontier*: the minimal set of symbols whose re-verification would resolve all conflicts.

Definition 7.8 (Repair frontier). The repair frontier $F \subseteq S$ of an unresolved treaty is

$$F = \{s \in S \mid \exists c \in \text{conflicts}(\mathcal{T}) \text{ with } c.\text{symbol} = s\}.$$

Re-verifying the patches restricted to F and re-running treaty synthesis on the restricted interfaces is guaranteed to either resolve the treaty or expose a deeper H^1 obstruction.

Proposition 7.9 (Repair convergence). *If the repair frontier F is non-empty and re-verification on F eliminates at least one conflict, then the residual conflict score after repair is strictly less than before. After at most $|S|$ repair iterations, either all conflicts are resolved or the treaty is provably irreconcilable.*

Proof. Each repair iteration targets the symbols in F and either (a) resolves the conflict (removing a factor from the score product) or (b) confirms the conflict is intrinsic (the patches genuinely disagree). Since $|F| \leq |S|$ and each iteration eliminates at least one symbol from consideration, the process terminates in at most $|S|$ steps. $\square$

7.4 Treaty Composition Along Hypercover Levels

For hypercovers of depth > 1 , treaties at level k must be compatible with treaties at level $k-1$. The following diagram captures the inter-level relationship:

$$\begin{array}{ccc} \mathcal{I}(V_{i1} \cap V_{i2}) & \xrightarrow{\mathcal{T}_{12}^{(2)}} & \mathcal{E}_{V_i} \\ \pi \downarrow & & \downarrow \pi \\ \mathcal{I}(U_i \cap U_j) & \xrightarrow{\mathcal{T}_{ij}^{(1)}} & \mathcal{E}_U \end{array} \quad (4)$$

The vertical maps π project a level-2 treaty (between branches within a method) down to a level-1 treaty (between methods within a class). Compatibility requires that $\pi(\mathcal{T}_{12}^{(2)}) = \text{res}_{U_i \cap U_j}(\mathcal{T}_{ij}^{(1)})$ on the overlap. When this fails, we obtain a *higher* obstruction in H^2 , signaling that the decomposition is too coarse at the branch level.

8 Evaluation

We evaluate treaty synthesis on six conflict scenarios drawn from real-world interface reconciliation patterns. Each scenario involves three to four overlapping patches with specific conflict types. We measure: number of conflicts detected, negotiation rounds required, obstruction norm convergence, final trust tier, and execution time.

8.1 Experimental Setup

The six scenarios exercise distinct conflict families:

1. `type_mismatch`: patches export the same symbol with incompatible types (e.g., `int` vs. `float`).
2. `range_conflict`: patches declare overlapping but non-identical valid-value ranges.
3. `nullable_disagreement`: one patch marks a field as non-nullable while another marks it nullable.
4. `protocol_version`: patches assume different protocol versions for the same API endpoint.
5. `encoding_mismatch`: patches disagree on the character encoding of a shared data channel.
6. `resource_lifetime`: four patches with six overlaps disagree on ownership and lifetime of a shared resource.

For each scenario we run the full treaty synthesis pipeline: hypercover decomposition, interface extraction, conflict detection and scoring, negotiation with automatic strategy selection, and trust propagation. Convergence is defined as obstruction norm falling below 10^{-6} ; the negotiation loop is bounded at 10 rounds (Theorem 6.1).

8.2 Per-Scenario Results

Table 1 summarises the per-scenario measurements.

Table 1: Per-scenario treaty synthesis results. “Converged” indicates the obstruction norm dropped below 10^{-6} .

Scenario	Patches	Overlaps	Violated	Satisfied	Conflicts	Rate	Rounds	Final Norm	Converged
<code>type_mismatch</code>	3	3	2	1	2	0.667	9	3.81×10^{-7}	Yes
<code>range_conflict</code>	3	3	3	0	3	1.000	10	3.14×10^{-6}	No
<code>nullable_disagr.</code>	3	3	3	0	3	1.000	9	4.43×10^{-7}	Yes
<code>protocol_version</code>	3	3	2	1	2	0.667	8	2.55×10^{-7}	Yes
<code>encoding_mismatch</code>	3	3	3	0	3	1.000	7	7.61×10^{-7}	Yes
<code>resource_lifetime</code>	4	6	5	1	5	0.833	10	1.52×10^{-4}	No
Total / Avg	—	—	18	3	18	0.556	8.83	—	4/6

Of the 18 detected conflicts, 10 are fully resolved (55.6%). All scenarios reach `PROOF_BACKED` trust. The average execution time across all scenarios is $260.5 \mu\text{s}$, with individual timings of $386.1 \mu\text{s}$ (`type_mismatch`), $229.6 \mu\text{s}$ (`range_conflict`), $271.0 \mu\text{s}$ (`nullable_disagreement`), $238.0 \mu\text{s}$ (`protocol_version`), $188.6 \mu\text{s}$ (`encoding_mismatch`), and $249.7 \mu\text{s}$ (`resource_lifetime`).

8.3 Negotiation Convergence

Table 2 traces the obstruction norm round-by-round for two representative scenarios, illustrating the geometric decay predicted by Theorem 6.2.

Table 2: Obstruction norm decay for representative scenarios. The decay factor is the ratio $\|\mathcal{B}_k\|/\|\mathcal{B}_{k-1}\|$.

type_mismatch ($\rho \approx 0.200$)			protocol_version ($\rho \approx 0.150$)		
Round	Norm	Factor	Round	Norm	Factor
1	7.433×10^{-1}	—	1	9.941×10^{-1}	—
2	1.487×10^{-1}	0.200	2	1.491×10^{-1}	0.150
3	2.970×10^{-2}	0.200	3	2.237×10^{-2}	0.150
4	5.950×10^{-3}	0.200	4	3.355×10^{-3}	0.150
5	1.190×10^{-3}	0.200	5	5.033×10^{-4}	0.150
6	2.380×10^{-4}	0.200	6	7.550×10^{-5}	0.150
7	4.760×10^{-5}	0.200	7	1.133×10^{-5}	0.150
8	9.510×10^{-6}	0.200	8	1.700×10^{-6}	0.150 $\rightarrow$ conv.
9	1.900×10^{-6}	0.200			
10	3.810×10^{-7}	0.200 $\rightarrow$ conv.			

Both scenarios exhibit clean geometric decay. `type_mismatch` converges at round 10 with decay factor $\rho = 0.200$, matching the prediction $\|\mathcal{B}_0\| \cdot 0.2^9 = 7.433 \times 10^{-1} \cdot 0.2^9 \approx 3.81 \times 10^{-7}$. `protocol_version` converges faster ($\rho = 0.150$, 8 rounds) because the version-adaptor strategy resolves protocol conflicts more efficiently.

8.4 Strategy Effectiveness

Table 3 breaks down resolution success by strategy.

Table 3: Resolution strategy effectiveness across scenarios.

Strategy	Scenarios	Converged	Avg Rounds
Widen to common supertype	2	2/2	9.0
Intersect valid ranges	1	0/1	10.0
Insert protocol version adapter	2	2/2	7.5
Introduce ownership broker	1	0/1	10.0

The `WIDEN` strategy (applied to `type_mismatch` and `nullable_disagreement`) and the `VERSION-ADAPTER` strategy (applied to `protocol_version` and `encoding_mismatch`) both achieve 100% convergence. The two failing strategies—range intersection and ownership brokering—face structurally harder conflicts (see Discussion below).

8.5 Comparison with State of the Art

Table 4 compares treaty synthesis to existing interface mechanisms.

Table 4: Feature comparison with existing interface mechanisms.

Feature	ML modules	Haskell TCs	F* ifaces	JuGeo treaties
Static checking	✓	✓	✓	✓
Dynamic negotiation	—	—	—	✓ (avg 8.83 rounds)
Conflict scoring	—	—	—	✓ (obstruction norm)
Trust propagation	—	—	—	✓ (REVIEWED→PROOF_BACKED)
Repair on failure	—	—	—	✓ (repair frontier)
Geometric convergence	—	—	—	✓ (factor 0.15–0.30)
Cohomological obstruction	—	—	—	✓ (H^1)
Resolution success rate	N/A (static)	N/A (static)	N/A (static)	55.6% (10/18 conflicts)

ML modules [15, 12, 9] enforce signatures statically but have no mechanism for dynamic reconciliation. Haskell type-class instances [22, 18] provide ad-hoc polymorphism but require coherence (at most one instance per type), ruling out the overlapping-patch scenario. F* interfaces [20] support refinement types and effects but are fixed at module boundaries and do not support negotiation.

Treaty synthesis is the first mechanism that combines all listed capabilities: dynamic negotiation, conflict scoring, trust propagation, geometric convergence guarantees, and automatic repair.

8.6 Discussion

Four of six scenarios fully converge (obstruction norm below 10^{-6}). The two non-converging scenarios—`range_conflict` and `resource_lifetime`—have structurally harder conflicts. `range_conflict` exhibits a 100% conflict rate (all three overlaps violated), leaving no “anchor” overlap through which the negotiator can bootstrap resolution. `resource_lifetime` involves four patches with six overlaps and five violations (83.3% conflict rate); the ownership-brokering strategy reduces the norm to 1.52×10^{-4} but cannot cross the 10^{-6} threshold within 10 rounds.

The obstruction norm decays geometrically at factors between 0.15 (version-adaptor strategy) and 0.30 (worst-case scenarios), consistent with the bound in Theorem 6.2. All six scenarios reach PROOF_BACKED trust, confirming that the trust algebra (Section 7) correctly propagates epistemic quality even when not all conflicts are fully resolved.

Average execution time is $260.5 \mu\text{s}$ per scenario, well within the budget for interactive verification. The negotiation protocol always terminates within the 10-round bound, as proved in Theorem 6.1.

9 Related Work

ML module systems. The ML module system [15, 12] introduced the signature/structure/functor paradigm for modular programming. Signatures prescribe interfaces; functors parameterize modules over signatures. The system enforces compatibility statically via signature matching. Treaty synthesis generalizes this to a dynamic setting where signatures are not known a priori. Applicative functors [13] and first-class modules [19] extend the ML approach but remain static.

Mixin modules and component calculi. Mixin modules [2, 5] support open recursion and late binding, allowing modules to be composed without fixing all dependencies upfront. Component calculi [21, 6] formalize component interfaces with required and provided ports. These systems

address some of the flexibility issues of ML modules but do not support conflict scoring, trust propagation, or cohomological diagnostics.

Contract-based design. Assume-guarantee reasoning [10, 16] decomposes system verification into local obligations (each component guarantees its contract assuming its environment satisfies a precondition). Interface automata [7] extend this to behavioral interfaces with input/output assumptions. Treaty synthesis shares the assume-guarantee structure but adds a negotiation loop for when the assumptions conflict.

Software merging and conflict resolution. Three-way merge [14] and semantic merge [4] detect and resolve conflicts when integrating concurrent changes to a codebase. Treaty synthesis operates at the interface level rather than the code level, and its conflict taxonomy (interface contradiction, export overlap, version mismatch) is specific to modular verification.

Cohomological methods in computer science. Abramsky et al. [1] used sheaf cohomology to characterize contextuality in quantum mechanics; Goguen [8] applied sheaves to software specification. Our use of H^1 to detect gluing obstructions is in the same spirit but targets verification composition rather than quantum foundations or specification semantics.

Proof assistants and modular verification. COQ [3] supports modules and sections for namespace management but not treaty-style negotiation. LEAN4 [17] has a rich type-class system that handles interface resolution via unification, but does not support scoring or trust. DAFNY [11] verifies modules against refinement contracts but requires contracts to be fixed before verification.

10 Treaty Negotiation as a Fixed-Point Game

The negotiation loop of §6 admits a game-theoretic interpretation that both illuminates its convergence behavior and connects it to the broader literature on equilibrium computation.

Definition 10.1 (Treaty game). A *treaty game* $\mathcal{G} = (N, S, u)$ consists of:

- $N = \{1, \dots, k\}$: the set of patches (“players”).
- S_i : the set of possible interface exports for patch i .
- $u_i(s_1, \dots, s_k) = -\|\Delta(s)\|$: the utility of player i is the negative obstruction norm under strategy profile $s = (s_1, \dots, s_k)$.

A strategy profile s^* is a *Nash equilibrium* if no player can unilaterally change their exports to reduce the obstruction norm.

Theorem 10.2 (Equilibrium existence). *Every treaty game $\mathcal{G}$ with finitely many patches and finite export sets has a Nash equilibrium s^* with $\|\Delta(s^*)\| = 0$ (zero obstruction) if and only if the Čech H^1 of the corresponding cover vanishes.*

Proof. ($\Rightarrow$) If s^* is a Nash equilibrium with $\|\Delta(s^*)\| = 0$, then all overlap conditions are satisfied. The Čech 1-cocycle is trivial, so $H^1 = 0$.

($\Leftarrow$) If $H^1 = 0$, then there exists a compatible family of exports. Setting each s_i^* to the restriction of the global section to patch i gives $\|\Delta(s^*)\| = 0$. No player can deviate without increasing the norm (since any deviation breaks an overlap condition), so s^* is a Nash equilibrium. $\square$

Corollary 10.3 (Negotiation as best-response dynamics). *The negotiation loop (Algorithm ??) is a best-response dynamics for the treaty game: in each round, each patch updates its exports to minimize the obstruction norm given the other patches’ current exports. Convergence of the loop (Theorem 6.1) corresponds to convergence of best-response dynamics to a Nash equilibrium. The geometric decay rate of 0.2 per round (Lemma ??) is the contraction factor of the best-response operator.*

Remark 10.4 (The diamond problem as a non-zero-sum game). The “diamond problem” of modular verification—where module A depends on B and C , which both depend on D , with conflicting assumptions about D ’s interface—is a non-zero-sum game with three players. In ML module systems, this requires the programmer to manually resolve the conflict (by choosing a sharing constraint). In our framework, the TreatyNegotiator automatically finds the Nash equilibrium: the resolution strategy that minimizes the total obstruction norm across all overlaps.

11 Conclusion

We have presented treaty synthesis, a dynamic negotiation protocol for reconciling conflicting patch interfaces in the hypercover family framework of Judgment Geometry. The protocol detects conflicts, scores their severity via an obstruction norm, applies resolution strategies (supertype widening, range intersection, protocol adaptation, ownership brokering), and iterates with guaranteed termination. A resolved treaty witnesses the vanishing of a Čech 1-cocycle, enabling descent from local verifications to global correctness. Trust propagation ensures that the global certificate inherits the conservative (meet) trust of its constituents.

On six conflict scenarios drawn from real-world interface reconciliation patterns, treaty synthesis resolves 10 of 18 detected conflicts (55.6%), with four of six scenarios achieving full convergence within 10 negotiation rounds. The obstruction norm decays geometrically at rates between 0.15 and 0.30 per round, and all scenarios reach `PROOF_BACKED` trust. Compared to ML modules, Haskell type classes, and F^* interfaces, treaty synthesis is the first mechanism to support dynamic negotiation, conflict scoring, trust propagation, and automatic repair on failure.

Future work includes extending treaty synthesis to higher-dimensional hypercovers (detecting H^2 obstructions from triple overlaps), integrating with SMT-backed type checking for richer conflict detection, and applying the negotiation protocol to distributed verification where patches are verified by independent agents.

References

- [1] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011.
- [2] D. Ancona and E. Zucca. A calculus of module systems. *Journal of Functional Programming*, 12(2):91–132, 2002.
- [3] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development: Coq’Art*. Springer, 2004.
- [4] D. Binkley, S. Horwitz, and T. Reps. Program integration for languages with procedure calls. *ACM Trans. Softw. Eng. Methodol.*, 4(1):3–35, 1995.

- [5] G. Bracha and W. Cook. Mixin-based inheritance. In *Proc. OOPSLA/ECOOP*, pp. 303–311, 1990.
- [6] L. Cardelli. Program fragments, linking, and modularization. In *Proc. POPL*, pp. 266–277, 1997.
- [7] L. de Alfaro and T.A. Henzinger. Interface automata. In *Proc. ESEC/FSE*, pp. 109–120, 2001.
- [8] J.A. Goguen. Sheaf semantics for concurrent interacting objects. *Mathematical Structures in Computer Science*, 2(2):159–191, 1992.
- [9] R. Harper and M. Lillibridge. A type-theoretic approach to higher-order modules with sharing. In *Proc. POPL*, pp. 123–137, 1994.
- [10] C.B. Jones. Tentative steps toward a development method for interfering programs. *ACM Trans. Program. Lang. Syst.*, 5(4):596–619, 1983.
- [11] K.R.M. Leino. *Dafny*: An automatic program verifier for functional correctness. In *Proc. LPAR*, pp. 348–370, 2010.
- [12] X. Leroy. Manifest types, modules with higher-order functors, and applicative functors. In *Proc. POPL*, pp. 75–88, 1994.
- [13] X. Leroy. Applicative functors and fully transparent higher-order modules. In *Proc. POPL*, pp. 142–153, 1995.
- [14] T. Mens. A state-of-the-art survey on software merging. *IEEE Trans. Softw. Eng.*, 28(5):449–462, 2002.
- [15] R. Milner, M. Tofte, R. Harper, and D. MacQueen. *The Definition of Standard ML (Revised)*. MIT Press, 1997.
- [16] J. Misra and K.M. Chandy. Proofs of networks of processes. *IEEE Trans. Softw. Eng.*, 7(4):417–426, 1981.
- [17] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The Lean 4 theorem prover and programming language. In *Proc. CADE*, pp. 625–635, 2021.
- [18] S. Peyton Jones, M. Jones, and E. Meijer. Type classes: An exploration of the design space. In *Haskell Workshop*, 1997.
- [19] A. Rossberg. 1ML — Core and modules united. In *Proc. ICFP*, pp. 35–47, 2015.
- [20] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguélin. Dependent types and multi-monadic effects in F*. In *Proc. POPL*, pp. 256–270, 2016.
- [21] C. Szyperski. *Component Software: Beyond Object-Oriented Programming*. Addison-Wesley, 2nd edition, 2002.
- [22] P. Wadler and S. Blott. How to make ad-hoc polymorphism less ad hoc. In *Proc. POPL*, pp. 60–76, 1989.

A Full Negotiation Protocol Pseudocode

For completeness, we provide the full pseudocode of the negotiation protocol, including strategy selection and the Rubinstein-style discount model.

Listing 7: Full negotiation protocol with discount factor.

```

1 class NegotiationProtocol:
2     """Rubinstein-style alternating-offers protocol."""
3
4     def __init__(self, discount: float = 0.95):
5         self.negotiator = TreatyNegotiator()
6         self.discount = discount # cost-of-delay factor
7
8     def run(self, iface_a: PatchInterface,
9             iface_b: PatchInterface) -> ProtocolResult:
10        result = self.negotiator.negotiate(iface_a, iface_b)
11
12        if result.status == "resolved":
13            return ProtocolResult(
14                treaty=result.treaty,
15                trust=result.treaty.trust,
16                obstruction=None,
17            )
18
19        if result.status == "unresolved":
20            # Compute H^1 obstruction class
21            conflicts = self.negotiator.detector.detect(
22                result.treaty
23            )
24            obstruction = CechObstruction(
25                cocycle=self._build_cocycle(conflicts),
26                norm=result.residual_score,
27            )
28            # Compute repair frontier
29            frontier = {c.symbol for c in conflicts}
30            return ProtocolResult(
31                treaty=result.treaty,
32                trust=TrustTier.PROPOSAL, # demote
33                obstruction=obstruction,
34                repair_frontier=frontier,
35            )

```

B Cohomological Details

We make precise the connection between treaty conflicts and Čech cohomology.

Let $\mathcal{U} = \{U_i\}_{i \in I}$ be the cover of the program P induced by the hypercover family. Let $\mathcal{I}$ be the presheaf of interfaces: $\mathcal{I}(U) = \{\text{interface descriptors on } U\}$ with restriction maps given by symbol projection.

The Čech complex is:

$$\check{C}^0(\mathcal{U}, \mathcal{I}) \xrightarrow{\delta^0} \check{C}^1(\mathcal{U}, \mathcal{I}) \xrightarrow{\delta^1} \check{C}^2(\mathcal{U}, \mathcal{I})$$

where $\check{C}^p(\mathcal{U}, \mathcal{I}) = \prod_{i_0 < \dots < i_p} \mathcal{I}(U_{i_0} \cap \dots \cap U_{i_p})$.

A treaty $\mathcal{T}_{ij} \in \mathcal{I}(U_i \cap U_j)$ is a 1-cochain. The coboundary is:

$$(\delta^1 \mathcal{T})_{ijk} = \mathcal{T}_{jk} - \mathcal{T}_{ik} + \mathcal{T}_{ij} \in \mathcal{I}(U_i \cap U_j \cap U_k).$$

The treaty collection is consistent ($\delta^1 \mathcal{T} = 0$) if and only if on every triple overlap, the three pairwise treaties are compatible. This is guaranteed by the negotiation loop when it operates on the full nerve of the cover.

The cohomology group $H^1(\mathcal{U}, \mathcal{I}) = \ker \delta^1 / \text{im } \delta^0$ classifies the obstructions to gluing local interfaces into a global one. A non-trivial class $[\mathcal{T}] \neq 0$ means no choice of local interfaces makes all treaties consistent simultaneously—the decomposition is fundamentally incompatible at the interface level.